EX. NO : 7 Decision Tree implementation with K – mean Algorithm

Date :29.09.2025

Aim:

To implement the K-Means clustering algorithm in Python to partition a given dataset into K clusters by iteratively updating centroids and cluster assignments.

Procedure:

1. **Initialize centroids:**
   Randomly select K data points from the dataset as the initial centroids.
2. **Assign data points to clusters:**
   For each data point, compute the Euclidean distance to each centroid and assign it to the closest centroid's cluster.
3. **Update centroids:**
   For each cluster, calculate the mean of all points assigned to it to obtain the new centroid.
4. **Check for convergence:**
   Repeat steps 2 and 3 until the centroids no longer change or until a maximum number of iterations is reached.
5. **Output the final centroids and clusters.**

Program Coding:

```
import random

import math


# Sample data points

data = [

    [1, 2], [2, 3], [3, 1],
```

```
    [8, 9], [9, 10], [10, 8]
]

# Number of clusters
K = 2
# Randomly initialize K centroids from the data
centroids = random.sample(data, K)

# Euclidean distance function
def euclidean(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

# K-Means algorithm
for iteration in range(10):  # limit to 10 iterations
    clusters = [[] for _ in range(K)]
    # Assign points to nearest centroid
    for point in data:
        distances = [euclidean(point, centroid) for centroid in centroids]
        closest_index = distances.index(min(distances))
        clusters[closest_index].append(point)
    # Update centroids
    new_centroids = []
    for cluster in clusters:
        if cluster:  # avoid division by zero
```

```
            x_coords = [p[0] for p in cluster]
            y_coords = [p[1] for p in cluster]
            new_centroid = [sum(x_coords)/len(x_coords), sum(y_coords)/len(y_coords)]
            new_centroids.append(new_centroid)
        else:
            new_centroids.append(random.choice(data))  # reassign randomly if empty
    # Check for convergence
    if new_centroids == centroids:
        break
    else:
        centroids = new_centroids
# Output results
print("Final centroids:")
for c in centroids:
    print(c)
print("\nClusters:")
for i, cluster in enumerate(clusters):
    print(f"Cluster {i+1}: {cluster}")
```

Output:

```
Final centroids:
[2.0, 2.0]
[9.0, 9.0]
Clusters:
Cluster 1: [[1, 2], [2, 3], [3, 1]]
```

Cluster 2: [[8, 9], [9, 10], [10, 8]]

Result:

Thus, the K-Means clustering algorithm was successfully implemented. The program clustered the given data points into two groups based on their Euclidean distance to the centroids, iteratively refining the centroids until convergence was achieved.